

RCP103 – Rapport de TP

TP02 - Création d'un simulateur à évènements discrets

BOURDAYA Amina, BOUTY Gabriel, CLERC Julien

25 mai 2026

Résumé

Ce TP a pour objectif de créer un programme permettant de simuler un système simple de type Client(s)/Serveur(s) asynchrones. Ce simulateur nous permettra de réaliser des mesures variées pour qualifier notre système (nombre instantané de requêtes, temps d'attente, etc.), sans avoir à mettre en œuvre concrètement ce dernier.

1 Description du système simulé

Comme évoqué en introduction, ce rapport présente la réalisation d'un simple simulateur à évènements discrets du type Client(s)/Serveur :

Un ensemble de x clients ($x \geq 1$), envoient des messages à un ensemble de n serveurs ($n \geq 1$). Un portail sert d'intermédiaire entre les clients et les serveurs. Compte tenu que les serveurs ne peuvent traiter qu'un message à la fois, le portail permet de temporiser les messages en attente de traitement dans une file de taille fixe K ($1 \leq K \leq \text{inf}$).

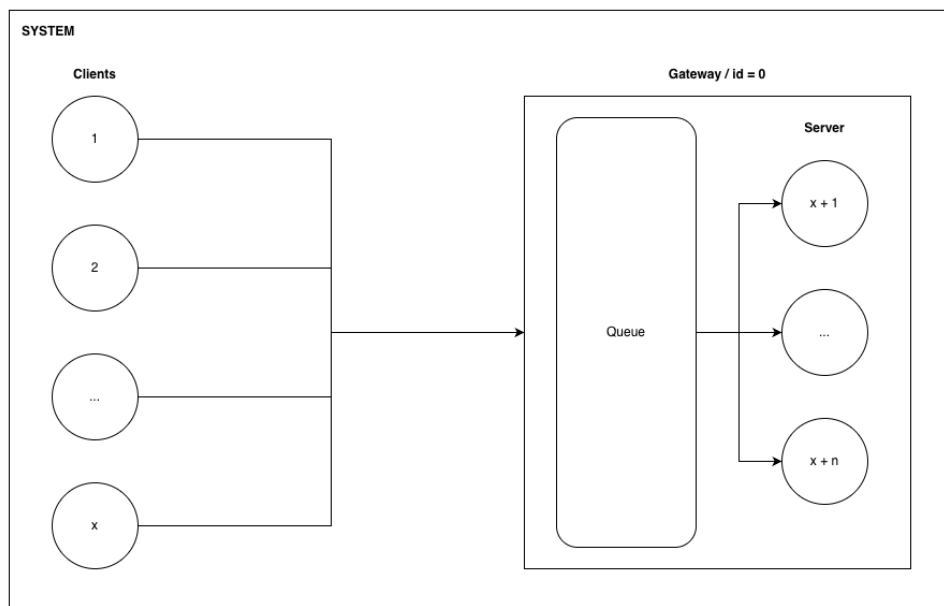


FIGURE 1 – Schéma du système simulé

Dans ce simulateur, les envois de messages suivent un processus de Poisson : le taux d'envois est constant dans le temps, les temps entre deux envois consécutifs suivent une distribution exponentielle. De même, le temps de service suit une loi exponentielle.

De manière générale, dans l'hypothèse où nous n'avons qu'un client, nous pouvons définir cette simulation comme un processus de file d'attente de type M/M/N/K :

- M : Arrivées selon un processus de Poisson (Markovien).
- M : Temps de service selon une distribution exponentielle (Markovien).
- N : Nombre de serveurs (ici, 1).
- K : Capacité du système (ici, la taille de la file du portail de capacité K).

La configuration du simulateur est définie comme suit :

- Nombre moyen de messages générés par unité de temps par client : λ
- Nombre moyen de messages traités par serveur par unité de temps : 8
- Temps de transmission constant : $1t$
- Temps total de la simulation : $t_{\max}$
- Nombre de clients : x
- Nombre de serveurs : n
- Taille de la file : k

2 Architecture du simulateur

2.1 Architecture initiale

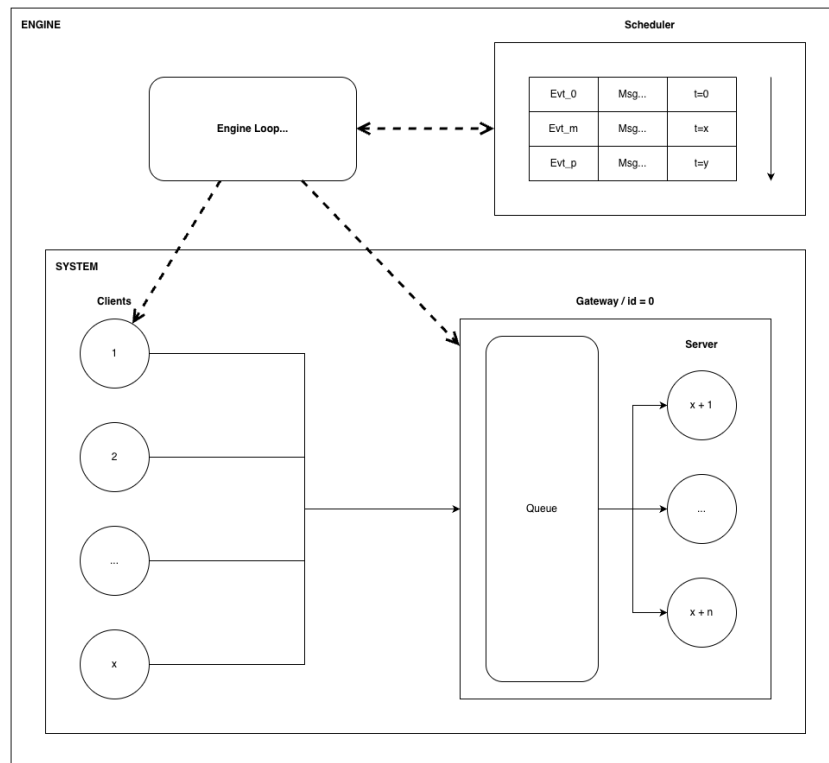


FIGURE 2 – Schéma de principe du simulateur

Afin de simuler les transmissions et traitement de messages entre le ou les *Client* et le *Server* (à travers le *Gateway*), chaque étape de gestion du *Message* sera encapsulée dans des *Event*. Ces *Event* pourront être, de fait, de 3 types différents :

- **SEND_MSG** : Le message est envoyé par le client.
- **RECV_MSG** : Le message est reçu par le portail.
- **MSG_DPT** : Le message est transmis au serveur (si ce dernier est disponible).

Chacun de ces *Event* sera timestampé et stocké chronologiquement dans un ordonnanceur (*Scheduler*). Ainsi, en ajoutant progressivement des événements à notre *Scheduler*, et en itérant sur ces derniers, nous pourrions simuler l'écoulement du temps, en nous concentrant uniquement sur les échantillons temporels qui nous intéressent.

L'ensemble de ces composants seront encapsulés dans un *Engine*, dont la boucle principale viendra interagir avec ces derniers afin de dérouler la simulation.

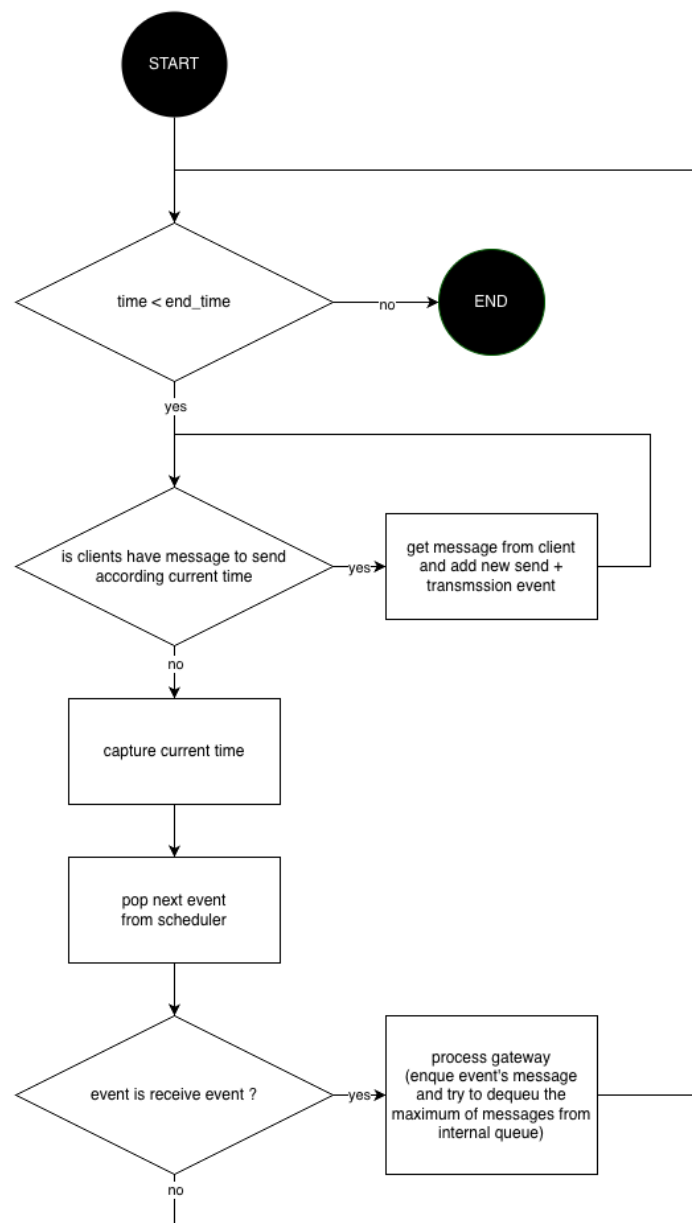


FIGURE 3 – Algorithme de la boucle principale de l'*Engine*

2.2 Organisation du code source

Le dossier fourni s'organise ainsi :

- **root** : Le point d'entrée du programme principal (`tp02_main.py`), les fichiers de configurations de lancement des tests, ainsi que les dépendances Python.
- Dossier **sources** : Le code applicatif.
- Dossier **tests** : Le code relatif aux tests unitaires.
- Dossiers **report** et **images** : Le présent rapport, et les résultats de la simulation, ainsi que les images associées au rapport.

3 Description des classes

3.1 Les données : class *Message* et class *Event*

Comme évoqué précédemment, la classe *Message* est l'unité d'échange entre *Client* et *Server*. Cette classe contient :

- L'identifiant du message : `message_id`
- L'identifiant du nœud émetteur : `source`
- L'identifiant du nœud destination : `destination`
- L'instant auquel le message est envoyé par le client : `send_time`
- L'instant auquel le message arrive à la passerelle : `arrival_time`
- L'instant auquel le traitement du message commence par le serveur : `server_start`

Ce *Message* est encapsulé dans un *Event*, dont les attributs sont les suivants :

- L'identifiant de l'évènement : `id`
- Le type de l'évènement : `type`
- Le *Message* encapsulé : `message`

Event possède par ailleurs un accesseur permettant de déterminer son timestamp, en fonction de son type.

3.2 L'ordonnanceur : class *Scheduler*

La classe *Scheduler* est chargée de gérer les futurs évènements du simulateur. Elle contient une liste d'évènements triés par ordre chronologique.

Lorsqu'un nouvel évènement est ajouté, la méthode parcourt la liste depuis la fin afin de trouver la bonne position d'insertion. Tant que le temps de l'évènement déjà présent est supérieur au temps du nouvel évènement, on remonte la liste. Lorsque la bonne position est trouvée, le nouvel évènement est inséré avec `insert`. Cela permet de garder la liste triée sans utiliser `sort` après chaque ajout.

La méthode `pop_event` retire et retourne le premier évènement de la liste. Comme la liste est maintenue triée par temps d'exécution, cet évènement correspond toujours au prochain évènement à traiter.

La méthode `get_current_time` retourne le temps du premier évènement de la liste, c'est-à-dire le temps courant du simulateur. Le *Scheduler* possède une mémoire du timestamp du dernier évènement retiré, afin de fournir toujours un temps de référence, même si sa liste d'évènement est vide.

La méthode `has_events` permet de savoir s'il reste encore des évènements à traiter.

Enfin, le *Scheduler* enregistre progressivement les évènements retirés, afin de pouvoir garder une trace de ces derniers.

3.3 Les composants simulés : class *Client* et *Server*

Les classes *Client* et *Server* représentent les composants principaux du système simulé. *Client* émet les messages qui seront traités par *Server*.

Comme décrit précédemment, les temps d'émissions comme les temps de traitement sont définis par une distribution exponentielle. Ainsi, on pourra définir pour chaque instances de *Client* et *Server*, une moyenne : pour *Client*, cette valeur correspond au nombre moyen de message émis par unité de temps, et pour *Server*, cette valeur correspond au nombre moyen de message traités par unité de temps.

Client expose quelques fonctions minimales :

- `get_next_msg_time` : permettant de connaître le timestamp du prochain envoi de *Message* émis par le *Client*.
- `pop_message` : permettant de simuler l'envoi du *Message*. Une instance préconfigurée de *Message* est renvoyée, et le timestamp interne est incrémenté d'une valeur aléatoire, générée selon une distribution exponentielle.

Server expose aussi quelques fonctions minimales :

- `is_free` : permettant de savoir si le *Server* est actuellement en train de traiter virtuellement un *Message*.
- `start_work` : permettant de simuler le début du traitement d'un *Message*. Un timestamp interne de fin de travail est défini selon le timestamp passé en argument, augmenté d'une valeur aléatoire, générée là aussi selon une distribution exponentielle.

3.4 La file d'attente des évènements : class *Queue*

La classe *Queue* est une structure de donnée de type FIFO (First In First Out). Elle va garder les messages dans l'attente de leur prise en compte par le serveur.

Voici le prototype et la description de chaque méthodes :

`def put(self, msg: Message)` : ajoute un message dans la file.

`def get(self)` : récupère le message le plus ancien ajouté dans la file.

`def is_empty(self)` : indique si la file est vide.

`def size(self)` : renvoie la taille de la file c'est-à-dire le nombre de message.

`def index(self, msg: Message)` : renvoie l'index d'un message dans la file. Cette méthode est utile pour les tests.

3.5 La passerelle : class *Gateway*

La classe *Gateway* est une classe dont le périmètre d'action est de gérer la file d'attente et les serveurs, elle manipule des messages. Elle teste la disponibilité des serveurs et envoie un message au premier serveur libre.

Voici le prototype et la description de la principale méthode :

`send_message(self, msg: Message, t_time: float)` : ajoute un message dans la file puis tant qu'un serveur est disponible, elle consomme les messages qui se trouvent dans cette même file. Elle renvoie une liste contenant les messages consommés.

3.6 Les fonctions de logs : fonctions Trace

Le module `trace` contient trois fonctions fournissant un système de journalisation au simulateur, permettant de visualiser et d'enregistrer les événements.

Voici le prototype et la description de chaque fonction :

`def get_time_and_node(e: Event)` : prend un événement en paramètre et retourne un tuple constitué du numéro de nœud et du temps en fonction du type d'évènement.

`def generateTraceOut(t_event: list[Event])` : prend une liste d'évènements en paramètre et écrit sur la sortie standard l'ensemble des événements contenu dans la liste.

`def generateTraceCSV(t_event: list[Event])` : prend une liste d'évènements en paramètre et écrit dans un fichier nommé `trace.csv` l'ensemble des événements contenu dans la liste au format CSV. Le délimiteur utilisé est le caractère `'`.

3.7 Le cœur du système : class Engine

La classe *Engine* représente le cœur du simulateur. Elle définit les instances des différents composants (*Client*, *Server*, etc.) lors de son initialisation, et simule les transactions entre chacun d'entre eux durant l'exécution de la simulation.

Sa fonction `run` permet de lancer l'exécution. Celle-ci s'appuie sur un *Scheduler* pour déterminer l'évolution du temps. Son rôle principale est de forcer les comportements de chacun des composants (*Client*, *Gateway*, etc.) et d'encapsuler les *Message* émis, reçu, traités dans des *Event*, afin d'alimenter le *Scheduler*.

La classe *Engine* propose aussi une fonction `log` permettant d'afficher les *Event* déjà passés, grâce aux fonctions proposées par le module *trace*.

3.8 Les statistiques : fonctions Stats

La fonction `average_time_in_queue(t_events: List[Event])` calcule le temps moyens que passe un message dans la file.

La fonction `maximum_waiting_time_in_queue(t_events: List[Event])` calcule le temps maximum que passe un message dans la file.

La fonction `total_messages_sended(t_events: List[Event], t_time: float)` calcule le nombre total de message envoyés par les clients à la gateway avant un temps donné.

La fonction `total_messages_received(t_events: List[Event], t_time: float)` calcule le nombre total de messages reçus par la gateway à un temps donné.

La fonction `total_messages_still_in_transmission(t_events: List[Event], t_time: float)` calcule le nombre total de messages en cours de transmission à un temps donné. Plus précisément il s'agit des messages qui ont été envoyés par les clients mais pas encore reçu par la gateway.

La fonction `total_messages_transmit(t_events: List[Event], t_time: float)` calcule le nombre total de messages transmis aux serveurs à un instant donné.

La fonction `messages_in_queue_at(t_events: List[Event], t_queue_size: int, t_time: float)` calcule le nombre total de messages dans la file à un instant donné.

La fonction `total_messages_dropped(t_events: List[Event], t_queue_size: int, t_time: float)` calcule le nombre total de messages droppés par la file à un instant donné.

La fonction `mean_queue_size(t_events: List[Event], t_queue_size: int, t_time: float)` calcule la taille moyenne de la file d'attente pour une période de temps donnée.

La fonction `rejection_rate(t_events: List[Event], t_queue_size: int, t_time: float)` calcule le taux des messages droppés (messages droppés / messages reçus).

3.9 Le point d'entrée : fonction main

Le point d'entrée du programme se trouve dans `tp02_main.py`. Il permet d'exécuter un programme qui utilise le simulateur avec différentes configurations. Le programme inscrit ensuite les résultats dans un rapport pdf. Le programme peut-être lancé simplement en appelant `python tp02_main.py` depuis le répertoire racine.

Les différentes configurations simulées sont :

- $\lambda = 4, x = 1, n = 1, k = \text{inf}$
- $\lambda = 6, x = 1, n = 1, k = \text{inf}$
- $\lambda = 8, x = 1, n = 1, k = \text{inf}$
- $\lambda = 12, x = 1, n = 1, k = \text{inf}$

- $\lambda = 4, x = 1, n = 1, k = 4$
- $\lambda = 6, x = 1, n = 1, k = 4$
- $\lambda = 8, x = 1, n = 1, k = 4$
- $\lambda = 12, x = 1, n = 1, k = 4$

- $\lambda = 4, x = 1, n = 1, k = 8$
- $\lambda = 6, x = 1, n = 1, k = 8$
- $\lambda = 8, x = 1, n = 1, k = 8$
- $\lambda = 12, x = 1, n = 1, k = 8$

- $\lambda = 4, x = 1, n = 3, k = 8$
- $\lambda = 6, x = 1, n = 3, k = 8$
- $\lambda = 8, x = 1, n = 3, k = 8$
- $\lambda = 12, x = 1, n = 3, k = 8$

Les résultats sont disponibles à la fin du présent rapport, ou directement dans le fichier de résultats : *simulation_results.pdf*

4 Tests unitaires

4.1 Le système de test : Pytest

Afin de vérifier le bon fonctionnement des classes implémentées, des tests unitaires ont été réalisés avec l'aide de l'utilitaire `Pytest`. L'objectif est de tester chaque classe séparément avant de l'intégrer dans le simulateur. L'ensemble des tests sont regroupés dans le répertoire `tests`.

4.2 Présentation des tests rédigés

Test de *Message* :

- Le test `test_message` vérifie que le constructeur initialise correctement les attributs du message (l'identifiant, la source et la destination, les différents timestamp).
- Le test `test_message_setters` vérifie que les setters modifient correctement les valeurs internes de l'objet, après modification de l'identifiant, de la source, de la destination, et des différents temps, les getters sont utilisés pour vérifier que les nouvelles valeurs ont bien été enregistrées.
- Le test `test_message_str` vérifie la méthode d'affichage de la classe *message*. Cette méthode est utile pour le débogage, car elle permet d'afficher rapidement les informations principales d'un message.

Test de *Event* :

- Le test `test_event_init` vérifie que le constructeur initialise correctement le temps de l'événement et son type. Un événement de type `SEND_MSG` est créé avec un temps égal à 1.5, puis les getters permettent de vérifier que ces valeurs sont bien stockées.
- Le test `test_event_setters` vérifie que le temps et le type d'un événement peuvent être modifiés correctement. Par exemple, un événement initialement de type `SEND_MSG` est modifié en `RECV_MSG`, et son temps est changé de 5.5 à 8.0.
- Le test `test_event_message_content` vérifie que l'événement conserve bien le message qui lui est associé. Après avoir récupéré le message depuis l'événement, le test vérifie que son identifiant, sa source et sa destination sont corrects.
- Le test `test_event_str` vérifie la méthode d'affichage de la classe *Event*. Cette méthode affiche l'identifiant de l'événement, son type, son temps et le message associé. Elle permet donc de faciliter le suivi des événements.

Test de *Server* :

- Le test `test_server` vérifie que la classe *Server* remplit bien son rôle de simulation de serveur, à savoir pouvoir démarrer un job, rester occupé pendant une période de temps aléatoire, puis être à nouveau libre à la fin de cette période de temps.

Test de *Client* :

- Le test `test_client_init` vérifie l'état initial d'un *Client*.
- Les tests `test_get_next_msg_time` et `test_pop_message` vérifient que *Client* renvoie bien un temps qui s'incrémente au fur et à mesure que les *Message* sont virtuellement émis par le *Client*.

Test de *Scheduler* :

- Le test `test_scheduler` vérifie que les temps des événements récupérés sont toujours dans l'ordre chronologique croissant. Cela permet de valider le rôle principal du *Scheduler* : maintenir les événements futurs triés par temps d'exécution. Le test vérifie aussi que les événements passés sont bien enregistrés dans la liste des événements passés.

Test de *Trace* :

- Le test défini par `test_get_time_and_node_event_SEND_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test défini par `test_get_time_and_node_event_RECV_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test défini par `test_get_time_and_node_event_MSG_DEPT` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test défini par `test_generateTraceOut_SEND_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.

- Le test définit par `test_generateTraceOut_RECV_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceOut_MSG_DEPT(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceCSV` vérifie que la fonction `generateTraceCSV` crée le fichier "trace.csv" avec les bonnes valeurs pour plusieurs messages.

Test de *Engine* :

- Le test `test_engine_complete` vérifie que l'exécution du `run` de *Engine* ne tombe pas dans une boucle sans fin. Il vérifie ensuite que les *Event* reçus suivent bien une logique temporelle incrémentale, et qu'ils soient bien cohérents les uns avec les autres. Par exemple, un `MSG_DEPT` doit avoir un `_RECV_MSG`, ainsi qu'un `SEND_MSG` correspondants avec des timestamps tel que $t_{MSG_DEPT} \geq t_{_RECV_MSG} \geq t_{SEND_MSG}$.

Test de *Queue* :

- Le test définit par `test_put` vérifie que la méthode `put` ajoute correctement un message.
- Le test définit par `test_get` vérifie que la méthode `get` renvoie correctement un message et le supprime de la file d'attente.
- Le test définit par `test_is_empty` vérifie que la méthode `is_empty` renvoie le booléen `True` si la file est vide
- Le test définit par `test_size` vérifie que la méthode `size` renvoie la taille de la file.
- Le test définit par `test_str` vérifie que la méthode `__str__` renvoie la chaîne de caractère comprenant l'ensemble des éléments de la file d'attente.

Test de *Gateway* :

- Le test définit par `test_send_message` vérifie que la méthode `send_message` envoie un message à un serveur libre.
- Le test définit par `test_send_message_dropped_or_in_queue` vérifie que la méthode `send_message` envoie un message à un serveur libre et que les messages suivants envoyés à la gateway sont soit droppés soit dans la queue dans le cas où aucun serveur n'est libre.

Test de *Stats* :

- Le test définit par `test_all_stats` vérifie que le bon fonctionnement de l'ensemble des fonctions de calculs des statistiques. Une simulation avec 2 clients, 2 serveurs et une capacité de 4 envois par unité de temps pendant 10 unités de temps est lancée. Ce test calcule d'abord le temps moyen et maximal passé dans la file, puis vérifie que ces valeurs sont non nulles. Ensuite, il calcule les statistiques détaillées (messages envoyés, transmis, abandonnés, en transmission ou en file) des événements.
- Le test définit par `test_all_stats_with_different_combination` vérifie que le bon fonctionnement de l'ensemble des fonctions de calculs des statistiques. Plusieurs simulations utilisant différentes combinaisons pendant 10 unités de temps sont lancées. Les paramètres possibles sont les suivants : une file de taille 4, 8, 12, 16, un nombre maximum de clients de 8 et un nombre maximum de serveurs de 8. Ce test calcule les statistiques détaillées (messages envoyés, transmis, abandonnés, en transmission ou en file) des événements. Il vérifie pour chaque simulation que le nombre de message envoyé est égal à l'addition des messages transmis, des messages abandonnés, des messages encore dans la file d'attente et des messages en cours de transmission.
- Le test définit par `test_mean_queue_size` calcule de la taille moyenne de la file d'attente sur une simulation de 10 unités de temps avec 2 clients, 2 serveurs et une file de taille 4. Il exécute la simulation, calcule la taille moyenne de la file sur toute la durée. Enfin, ce test que cette moyenne est bien comprise entre 0 et la taille maximale de la file.

- Le test défini par `test_mean_rqt_in_system` calcule du nombre moyen de requêtes dans le système sur une simulation de 10 unités de temps avec 2 clients, 2 serveurs et une file de taille 4. Cette simulation permet de calculer la moyenne à partir des événements et vérifie que cette moyenne est bien positive ou nulle.

4.3 Exécution des tests

Les tests peuvent être lancés depuis le répertoire principal en tapant la commande `pytest` (attention, `Pytest` doit être préalablement installé). L'ensemble des tests présents dans le répertoire `tests` seront exécutés.

5 Conclusion

Ce TP nous permet d'appréhender ce que peut être un simulateur à événements discrets. Nous mettons en œuvre des concepts vus en cours tels que les files d'attente ou encore les processus de Poisson.

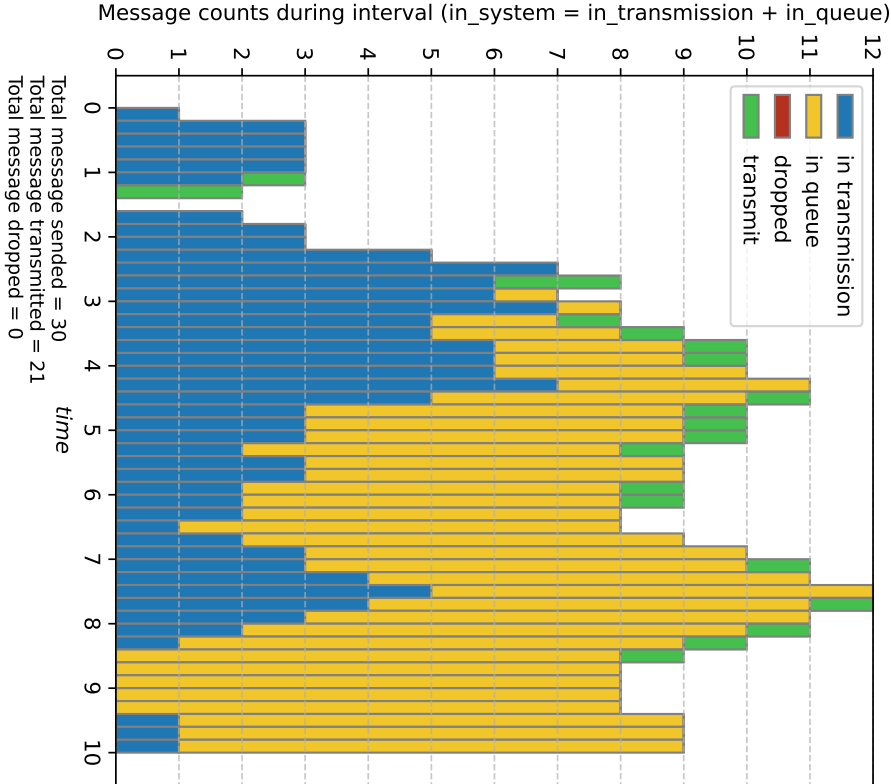
Il nous permet aussi de comprendre l'intérêt que peut représenter la complémentarité qu'il peut exister entre modélisation mathématique et simulation. En effet, la modélisation mathématique nous fournit les outils (variables aléatoires, processus stochastiques, etc.) qui vont nous permettre de construire facilement un simulateur, et ainsi réaliser, tout aussi facilement, des mesures théoriques sur un système complexe. Établir et mettre en œuvre un protocole de mesures sur ce même système dans le monde réel aurait été bien plus lourd et coûteux.

Grâce aux résultats générés par le programme de démonstration, nous constatons facilement que ce simulateur va nous permettre d'extraire les métriques qui vont nous guider dans la configuration d'un système. En fonction des spécifications d'acceptabilités définies (temps de traitement maximum ou moyen, taux de perte autorisé, ressources allouables), nous pourrions adapter la configuration du simulateur pour trouver le meilleur compromis, voir la configuration idéale, répondant à ces spécifications.

D'un point de vue du développement logiciel, les différentes itérations de réalisation, basées sur une approche TDD (Test Driven Development), nous permettent de valider progressivement nos composants logiciels. Cela facilite ensuite l'assemblage, et permet aussi d'éviter les régressions. Cependant, certaines améliorations seraient à envisager pour parfaire ce simulateur tel que :

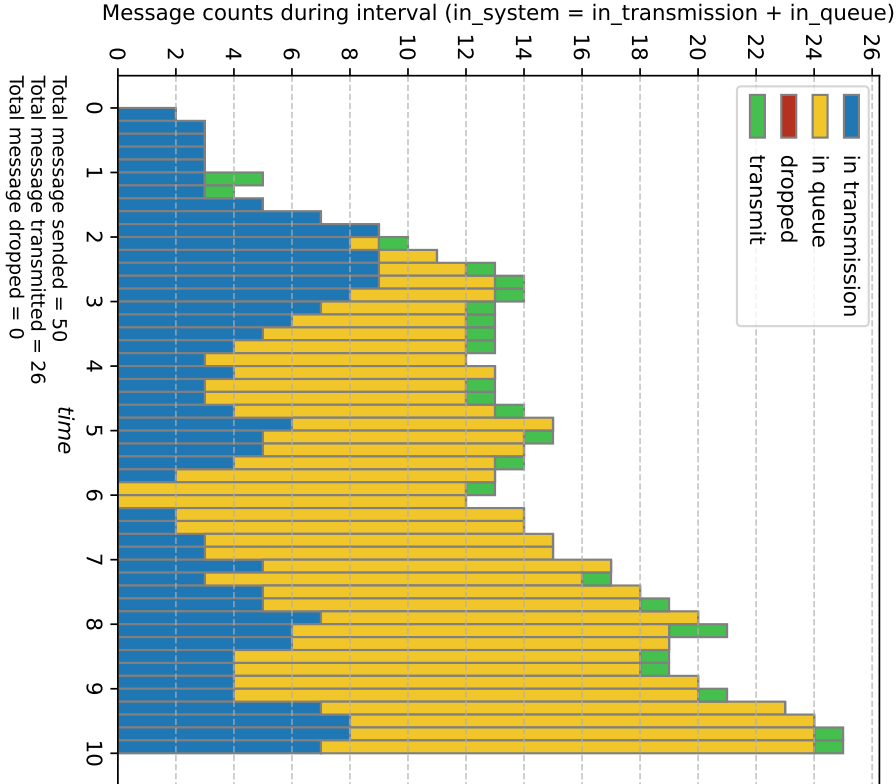
- L'amélioration des performances des fonctions statistiques.
- Rendre configurable le seed de génération de message (actuellement basé sur l'id du client uniquement)
- etc.

Duration=10, Clients count=1, Client's $\lambda=4$, Servers count=1, $Q=\text{inf}$.



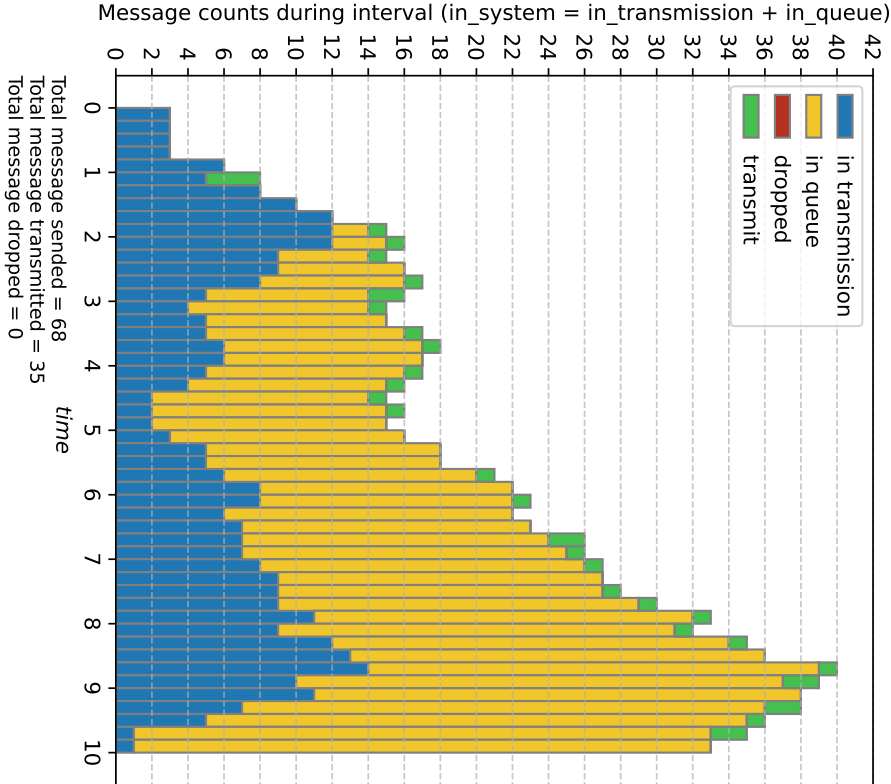
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.2683	1	SEND_MSG	1	0	1
0.3454	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.2683	0	RECV_MSG	1	0	1
1.2683	0	MSG_DEPT	0	2	1
1.3454	0	RECV_MSG	1	0	2
1.3454	0	MSG_DEPT	0	2	2
1.6892	1	SEND_MSG	1	0	3
1.7808	1	SEND_MSG	1	0	4
1.8097	1	SEND_MSG	1	0	5
2.2596	1	SEND_MSG	1	0	6
2.3843	1	SEND_MSG	1	0	7
2.5221	1	SEND_MSG	1	0	8
2.5296	1	SEND_MSG	1	0	9
2.6892	0	RECV_MSG	1	0	3
2.6892	0	MSG_DEPT	0	2	3
2.7206	1	SEND_MSG	1	0	10
2.7808	0	RECV_MSG	1	0	4
2.7808	0	MSG_DEPT	0	2	4
...	...	...	...	...	...
6.1329	1	SEND_MSG	1	0	23
6.1982	0	RECV_MSG	1	0	21
6.1982	0	MSG_DEPT	0	2	15
6.4229	0	RECV_MSG	1	0	22
6.6406	1	SEND_MSG	1	0	24
6.9321	1	SEND_MSG	1	0	25
7.0142	1	SEND_MSG	1	0	26
7.1329	0	RECV_MSG	1	0	23
7.1329	0	MSG_DEPT	0	2	16
7.3729	1	SEND_MSG	1	0	27
7.4123	1	SEND_MSG	1	0	28
7.6406	0	RECV_MSG	1	0	24
7.6406	0	MSG_DEPT	0	2	24
7.9321	0	RECV_MSG	1	0	25
8.0142	0	RECV_MSG	1	0	26
8.0142	0	MSG_DEPT	0	2	18
8.3729	0	RECV_MSG	1	0	27
8.3729	0	MSG_DEPT	0	2	19
8.4123	0	RECV_MSG	1	0	28
8.4123	0	MSG_DEPT	0	2	20
9.5181	1	SEND_MSG	1	0	29
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=6$, Servers count=1, $Q=\text{inf}$.



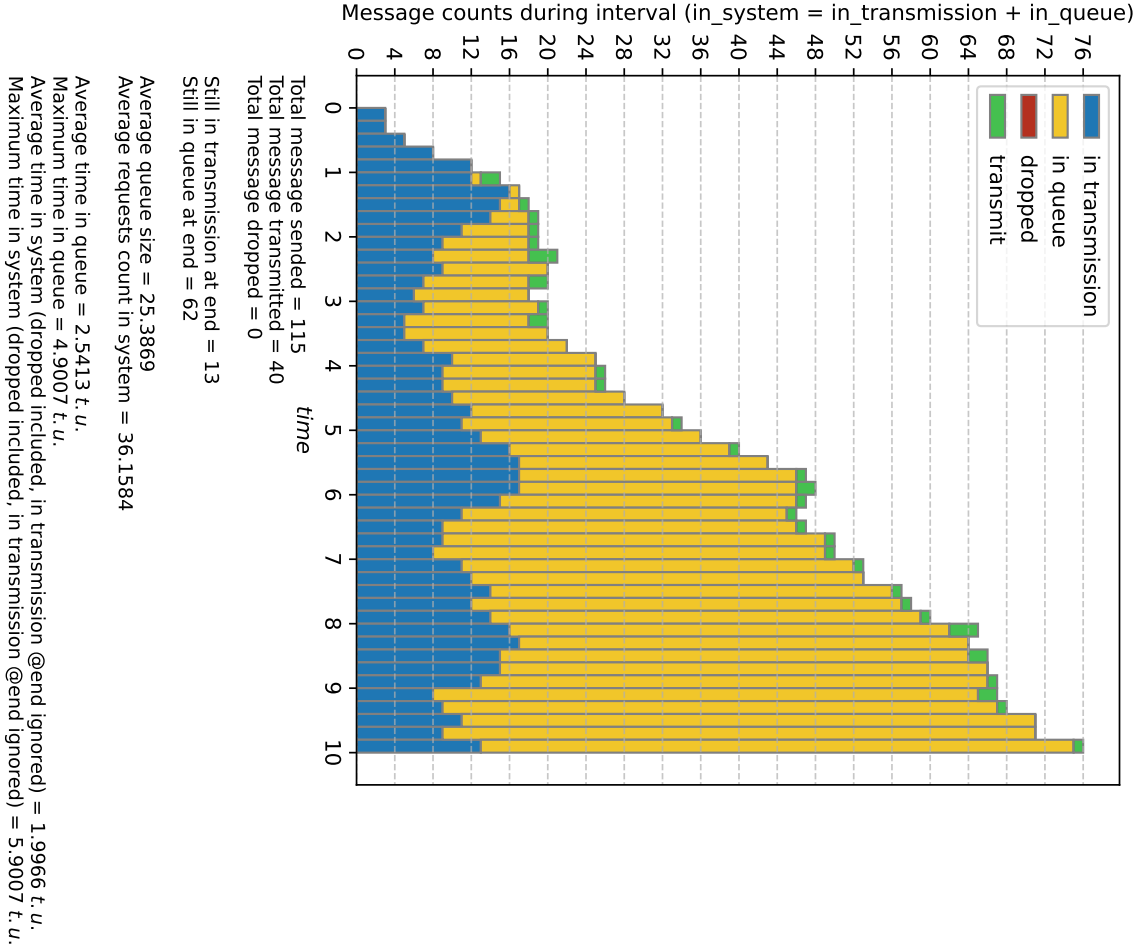
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1788	1	SEND_MSG	1	0	1
0.2302	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1262	1	SEND_MSG	1	0	3
1.1788	0	RECV_MSG	1	0	1
1.1788	0	MSG_DEPT	0	2	1
1.1872	1	SEND_MSG	1	0	4
1.2065	1	SEND_MSG	1	0	5
1.2302	0	RECV_MSG	1	0	2
1.2302	0	MSG_DEPT	0	2	2
1.5064	1	SEND_MSG	1	0	6
1.5895	1	SEND_MSG	1	0	7
1.6814	1	SEND_MSG	1	0	8
1.6864	1	SEND_MSG	1	0	9
1.8137	1	SEND_MSG	1	0	10
1.9873	1	SEND_MSG	1	0	11
2.0257	1	SEND_MSG	1	0	12
2.1262	0	RECV_MSG	1	0	3
2.1262	0	MSG_DEPT	0	2	3
...	...	...	...	...	...
8.7022	0	RECV_MSG	1	0	36
8.7022	0	MSG_DEPT	0	2	22
8.7215	1	SEND_MSG	1	0	40
8.855	1	SEND_MSG	1	0	41
8.9016	0	RECV_MSG	1	0	37
8.9115	0	RECV_MSG	1	0	38
8.9493	1	SEND_MSG	1	0	42
9.0503	1	SEND_MSG	1	0	43
9.0955	0	RECV_MSG	1	0	39
9.0955	0	MSG_DEPT	0	2	23
9.2447	1	SEND_MSG	1	0	44
9.3329	1	SEND_MSG	1	0	45
9.3776	1	SEND_MSG	1	0	46
9.5487	1	SEND_MSG	1	0	47
9.7215	0	RECV_MSG	1	0	40
9.7215	0	MSG_DEPT	0	2	24
9.7891	1	SEND_MSG	1	0	48
9.855	0	RECV_MSG	1	0	41
9.855	0	MSG_DEPT	0	2	25
9.9242	1	SEND_MSG	1	0	49
9.9493	0	RECV_MSG	1	0	42
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=8$, Servers count=1, $Q=\text{inf}$.



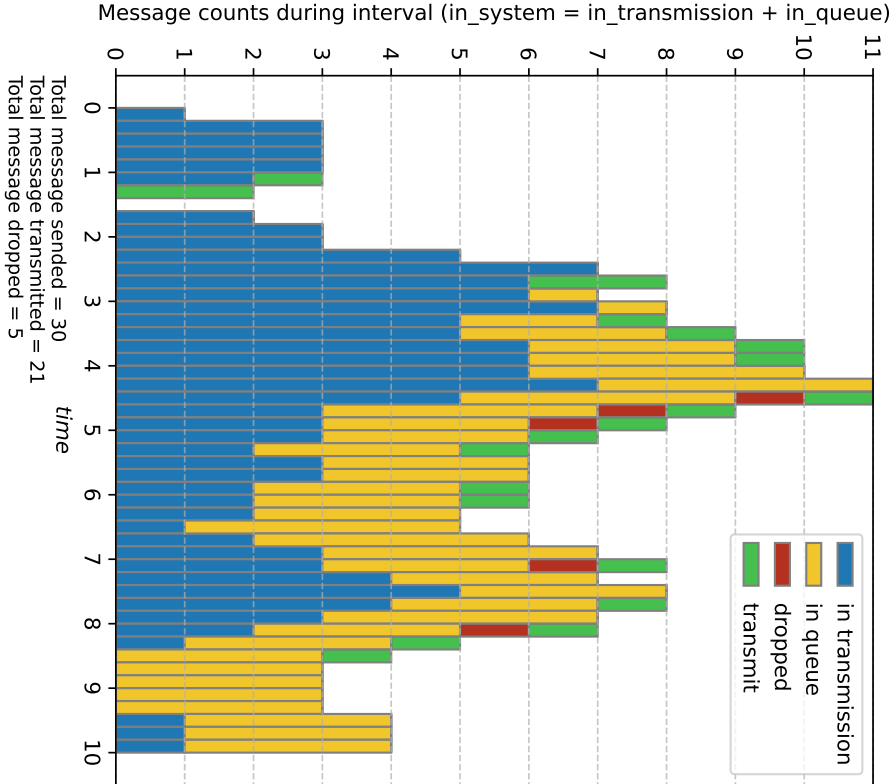
time	node	event	src	dst	msgid
0.0	1	SEND_MSG	1	0	0
0.1341	1	SEND_MSG	1	0	1
0.1727	1	SEND_MSG	1	0	2
0.8446	1	SEND_MSG	1	0	3
0.8904	1	SEND_MSG	1	0	4
0.9048	1	SEND_MSG	1	0	5
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1298	1	SEND_MSG	1	0	6
1.1341	0	RECV_MSG	1	0	1
1.1341	0	MSG_DEPT	0	2	1
1.1727	0	RECV_MSG	1	0	2
1.1727	0	MSG_DEPT	0	2	2
1.1921	1	SEND_MSG	1	0	7
1.2611	1	SEND_MSG	1	0	8
1.2648	1	SEND_MSG	1	0	9
1.3603	1	SEND_MSG	1	0	10
1.4905	1	SEND_MSG	1	0	11
1.5193	1	SEND_MSG	1	0	12
1.729	1	SEND_MSG	1	0	13
1.755	1	SEND_MSG	1	0	14
...	...	...	...	...	...
8.8039	0	RECV_MSG	1	0	54
8.8811	0	RECV_MSG	1	0	55
8.8811	0	MSG_DEPT	0	2	28
8.9902	0	RECV_MSG	1	0	56
8.9902	0	MSG_DEPT	0	2	29
9.1143	1	SEND_MSG	1	0	67
9.2526	0	RECV_MSG	1	0	57
9.2526	0	MSG_DEPT	0	2	30
9.3404	0	RECV_MSG	1	0	58
9.3404	0	MSG_DEPT	0	2	31
9.3674	0	RECV_MSG	1	0	59
9.3683	0	RECV_MSG	1	0	60
9.4018	0	RECV_MSG	1	0	61
9.4018	0	MSG_DEPT	0	2	32
9.4668	0	RECV_MSG	1	0	62
9.6223	0	RECV_MSG	1	0	63
9.6223	0	MSG_DEPT	0	2	33
9.6569	0	RECV_MSG	1	0	64
9.7562	0	RECV_MSG	1	0	65
9.7562	0	MSG_DEPT	0	2	34
9.7959	0	RECV_MSG	1	0	66
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =12, Servers count=1, Q=inf.



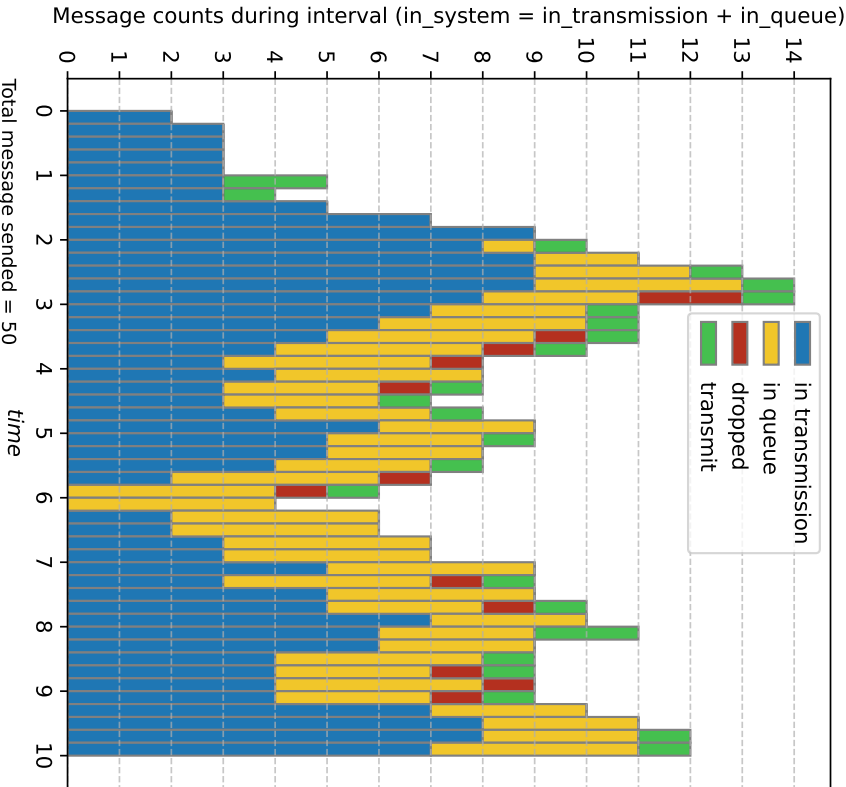
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.0894	1	SEND_MSG	1	0	1
0.1151	1	SEND_MSG	1	0	2
0.5631	1	SEND_MSG	1	0	3
0.5936	1	SEND_MSG	1	0	4
0.6032	1	SEND_MSG	1	0	5
0.7532	1	SEND_MSG	1	0	6
0.7948	1	SEND_MSG	1	0	7
0.8407	1	SEND_MSG	1	0	8
0.8432	1	SEND_MSG	1	0	9
0.9069	1	SEND_MSG	1	0	10
0.9937	1	SEND_MSG	1	0	11
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.0129	1	SEND_MSG	1	0	12
1.0894	0	RECV_MSG	1	0	1
1.0894	0	MSG_DEPT	0	2	1
1.1151	0	RECV_MSG	1	0	2
1.1526	1	SEND_MSG	1	0	13
1.17	1	SEND_MSG	1	0	14
1.2355	1	SEND_MSG	1	0	15
...	...	...	...	...	...
9.2051	1	SEND_MSG	1	0	103
9.2854	1	SEND_MSG	1	0	104
9.2959	0	RECV_MSG	1	0	95
9.2959	0	MSG_DEPT	0	2	38
9.2981	1	SEND_MSG	1	0	105
9.327	0	RECV_MSG	1	0	96
9.4192	0	RECV_MSG	1	0	97
9.4208	1	SEND_MSG	1	0	106
9.4393	1	SEND_MSG	1	0	107
9.4638	1	SEND_MSG	1	0	108
9.5373	1	SEND_MSG	1	0	109
9.5839	0	RECV_MSG	1	0	98
9.6362	0	RECV_MSG	1	0	99
9.6499	0	RECV_MSG	1	0	100
9.8514	1	SEND_MSG	1	0	110
9.8721	1	SEND_MSG	1	0	111
9.9032	1	SEND_MSG	1	0	112
9.9139	1	SEND_MSG	1	0	113
9.9434	1	SEND_MSG	1	0	114
9.9484	0	RECV_MSG	1	0	101
9.9484	0	MSG_DEPT	0	2	39
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=4$, Servers count=1, Q=4



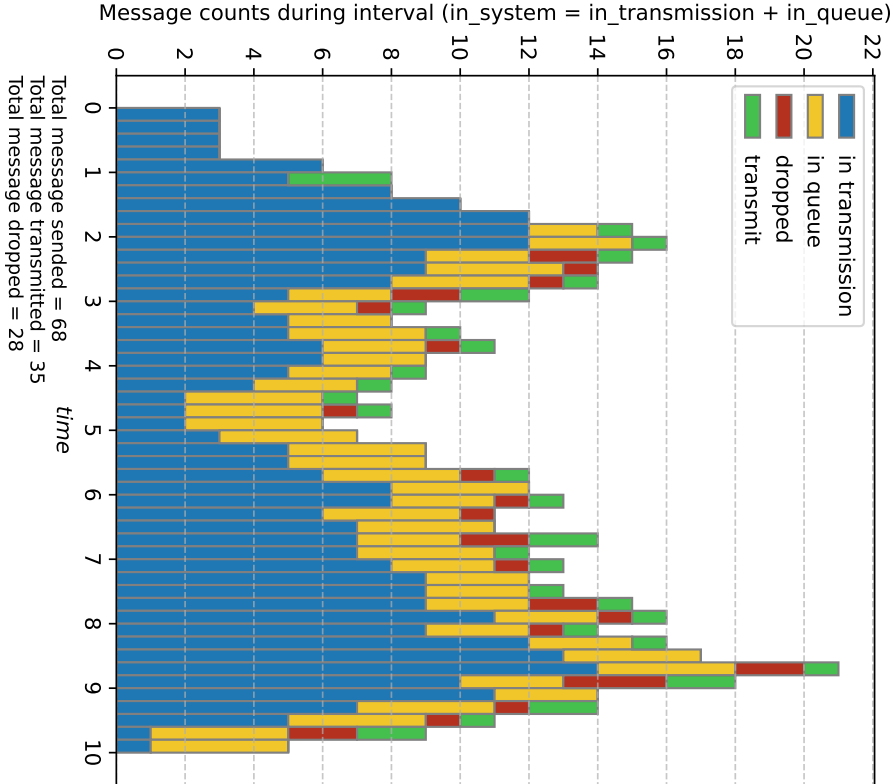
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.2683	1	SEND_MSG	1	0	1
0.3454	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.2683	0	RECV_MSG	1	0	1
1.2683	0	MSG_DEPT	0	2	1
1.3454	0	RECV_MSG	1	0	2
1.3454	0	MSG_DEPT	0	2	2
1.6892	1	SEND_MSG	1	0	3
1.7808	1	SEND_MSG	1	0	4
1.8097	1	SEND_MSG	1	0	5
2.2596	1	SEND_MSG	1	0	6
2.3843	1	SEND_MSG	1	0	7
2.5221	1	SEND_MSG	1	0	8
2.5296	1	SEND_MSG	1	0	9
2.6892	0	RECV_MSG	1	0	3
2.6892	0	MSG_DEPT	0	2	3
2.7206	1	SEND_MSG	1	0	10
2.7808	0	RECV_MSG	1	0	4
2.7808	0	MSG_DEPT	0	2	4
...	...	...	...	...	...
6.1329	1	SEND_MSG	1	0	23
6.1982	0	RECV_MSG	1	0	21
6.1982	0	MSG_DEPT	0	2	18
6.4229	0	RECV_MSG	1	0	22
6.6406	1	SEND_MSG	1	0	24
6.9321	1	SEND_MSG	1	0	25
7.0142	1	SEND_MSG	1	0	26
7.1329	0	RECV_MSG	1	0	23
7.1329	0	MSG_DEPT	0	2	19
7.3729	1	SEND_MSG	1	0	27
7.4123	1	SEND_MSG	1	0	28
7.6406	0	RECV_MSG	1	0	24
7.6406	0	MSG_DEPT	0	2	20
7.9321	0	RECV_MSG	1	0	25
8.0142	0	RECV_MSG	1	0	26
8.0142	0	MSG_DEPT	0	2	21
8.3729	0	RECV_MSG	1	0	27
8.3729	0	MSG_DEPT	0	2	22
8.4123	0	RECV_MSG	1	0	28
8.4123	0	MSG_DEPT	0	2	24
9.5181	1	SEND_MSG	1	0	29
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=6$, Servers count=1, $Q=4$



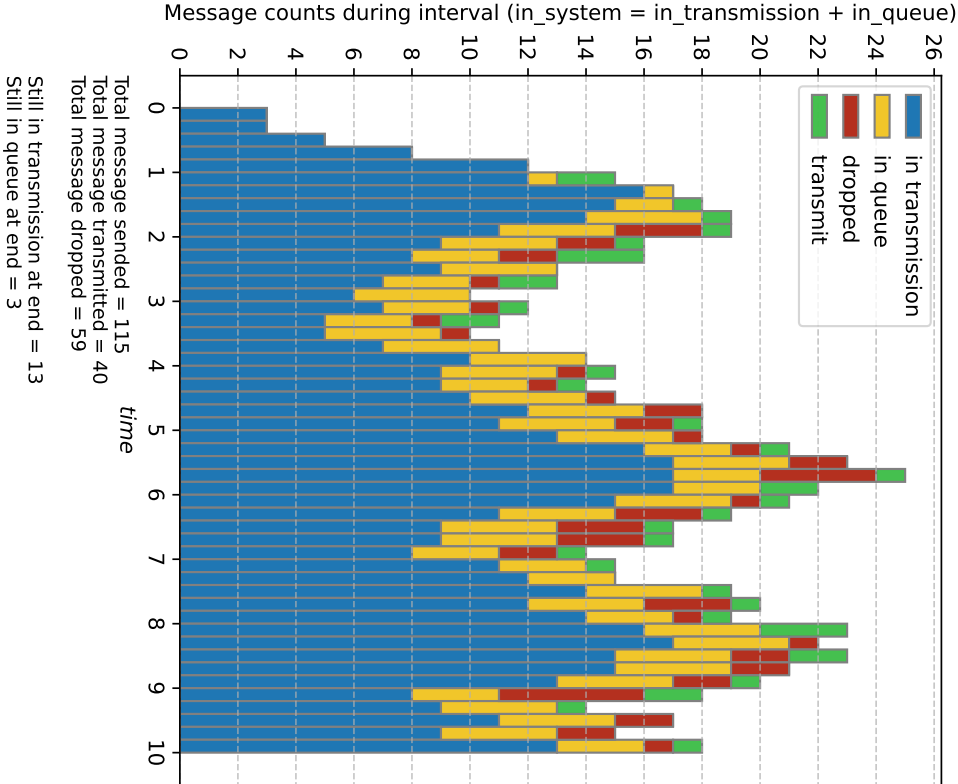
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1788	1	SEND_MSG	1	0	1
0.2302	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1262	1	SEND_MSG	1	0	3
1.1788	0	RECV_MSG	1	0	1
1.1788	0	MSG_DEPT	0	2	1
1.1872	1	SEND_MSG	1	0	4
1.2065	1	SEND_MSG	1	0	5
1.2302	0	RECV_MSG	1	0	2
1.2302	0	MSG_DEPT	0	2	2
1.5064	1	SEND_MSG	1	0	6
1.5895	1	SEND_MSG	1	0	7
1.6814	1	SEND_MSG	1	0	8
1.6864	1	SEND_MSG	1	0	9
1.8137	1	SEND_MSG	1	0	10
1.9873	1	SEND_MSG	1	0	11
2.0257	1	SEND_MSG	1	0	12
2.1262	0	RECV_MSG	1	0	3
2.1262	0	MSG_DEPT	0	2	3
...	...	...	...	...	...
8.7022	0	RECV_MSG	1	0	36
8.7022	0	MSG_DEPT	0	2	32
8.7215	1	SEND_MSG	1	0	40
8.855	1	SEND_MSG	1	0	41
8.9016	0	RECV_MSG	1	0	37
8.9115	0	RECV_MSG	1	0	38
8.9493	1	SEND_MSG	1	0	42
9.0503	1	SEND_MSG	1	0	43
9.0955	0	RECV_MSG	1	0	39
9.0955	0	MSG_DEPT	0	2	33
9.2447	1	SEND_MSG	1	0	44
9.3329	1	SEND_MSG	1	0	45
9.3776	1	SEND_MSG	1	0	46
9.5487	1	SEND_MSG	1	0	47
9.7215	0	RECV_MSG	1	0	40
9.7215	0	MSG_DEPT	0	2	34
9.7891	1	SEND_MSG	1	0	48
9.855	0	RECV_MSG	1	0	41
9.855	0	MSG_DEPT	0	2	35
9.9242	1	SEND_MSG	1	0	49
9.9493	0	RECV_MSG	1	0	42
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=8$, Servers count=1, Q=4



time	node	event	src	dst	msgid
0.0	1	SEND_MSG	1	0	0
0.1341	1	SEND_MSG	1	0	1
0.1727	1	SEND_MSG	1	0	2
0.8446	1	SEND_MSG	1	0	3
0.8904	1	SEND_MSG	1	0	4
0.9048	1	SEND_MSG	1	0	5
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1298	1	SEND_MSG	1	0	6
1.1341	0	RECV_MSG	1	0	1
1.1341	0	MSG_DEPT	0	2	1
1.1727	0	RECV_MSG	1	0	2
1.1727	0	MSG_DEPT	0	2	2
1.1921	1	SEND_MSG	1	0	7
1.2611	1	SEND_MSG	1	0	8
1.2648	1	SEND_MSG	1	0	9
1.3603	1	SEND_MSG	1	0	10
1.4905	1	SEND_MSG	1	0	11
1.5193	1	SEND_MSG	1	0	12
1.729	1	SEND_MSG	1	0	13
1.755	1	SEND_MSG	1	0	14
...	...	...	...	...	...
8.8039	0	RECV_MSG	1	0	54
8.8811	0	RECV_MSG	1	0	55
8.8811	0	MSG_DEPT	0	2	46
8.9902	0	RECV_MSG	1	0	56
8.9902	0	MSG_DEPT	0	2	48
9.1143	1	SEND_MSG	1	0	67
9.2526	0	RECV_MSG	1	0	57
9.2526	0	MSG_DEPT	0	2	49
9.3404	0	RECV_MSG	1	0	58
9.3404	0	MSG_DEPT	0	2	51
9.3674	0	RECV_MSG	1	0	59
9.3683	0	RECV_MSG	1	0	60
9.4018	0	RECV_MSG	1	0	61
9.4018	0	MSG_DEPT	0	2	56
9.4668	0	RECV_MSG	1	0	62
9.6223	0	RECV_MSG	1	0	63
9.6223	0	MSG_DEPT	0	2	57
9.6569	0	RECV_MSG	1	0	64
9.7562	0	RECV_MSG	1	0	65
9.7562	0	MSG_DEPT	0	2	58
9.7959	0	RECV_MSG	1	0	66
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =12, Servers count=1, Q=4

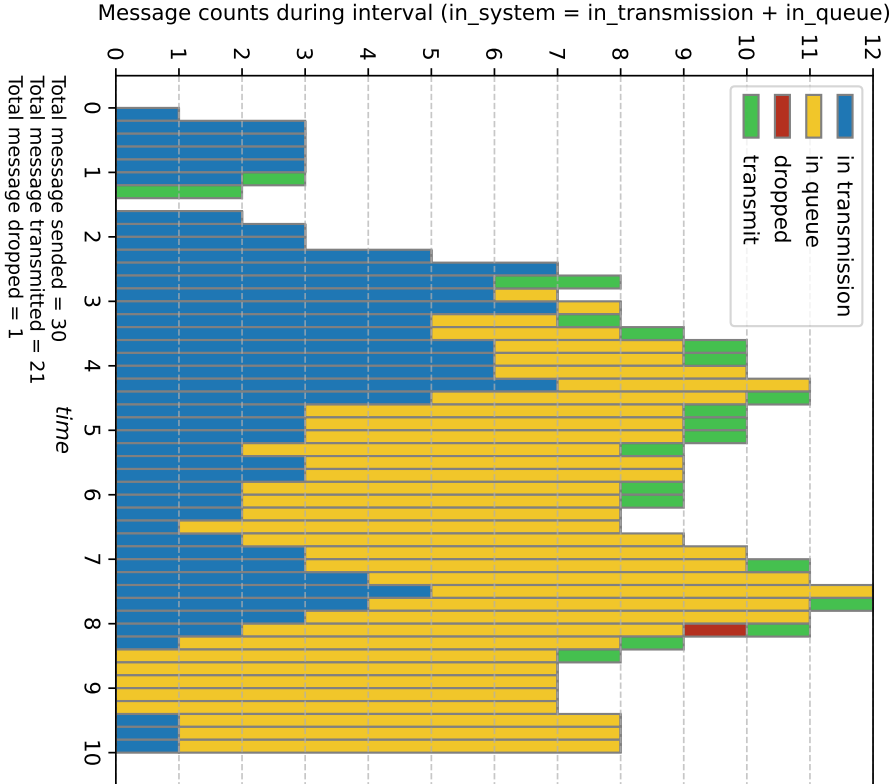


Average queue size = 3.0808
Average requests count in system = 13.8523

Average time in queue = 0.7124 t.u.
Maximum time in queue = 1.9031 t.u.
Average time in system (dropped included, in transmission @end ignored) = 1.2794 t.u.
Maximum time in system (dropped included, in transmission @end ignored) = 2.9031 t.u.

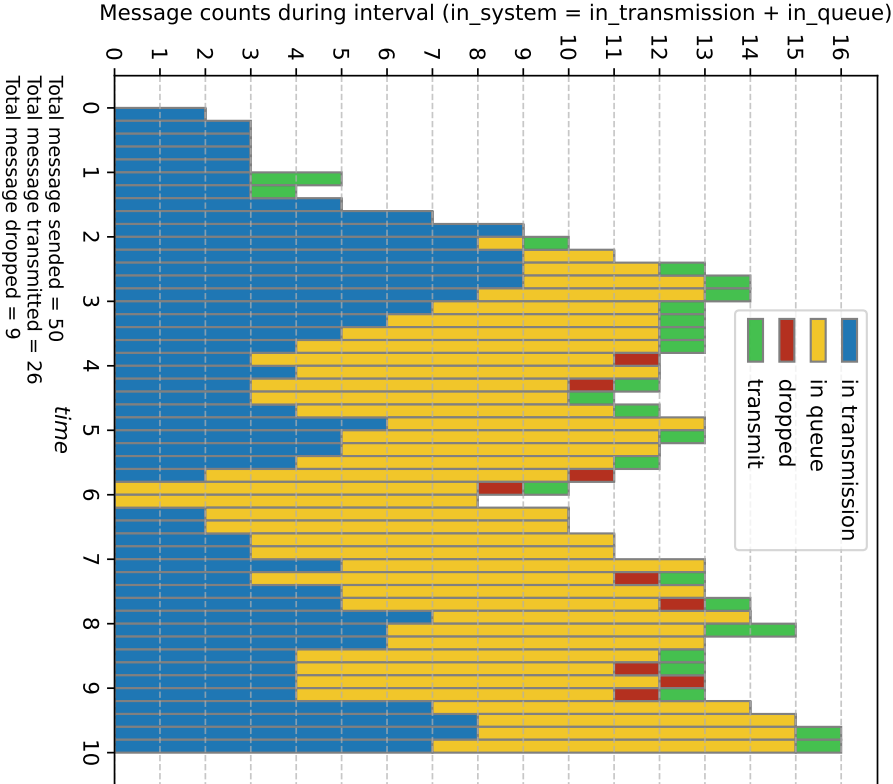
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.0894	1	SEND_MSG	1	0	1
0.1151	1	SEND_MSG	1	0	2
0.5631	1	SEND_MSG	1	0	3
0.5936	1	SEND_MSG	1	0	4
0.6032	1	SEND_MSG	1	0	5
0.7532	1	SEND_MSG	1	0	6
0.7948	1	SEND_MSG	1	0	7
0.8407	1	SEND_MSG	1	0	8
0.8432	1	SEND_MSG	1	0	9
0.9069	1	SEND_MSG	1	0	10
0.9937	1	SEND_MSG	1	0	11
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.0129	1	SEND_MSG	1	0	12
1.0894	0	RECV_MSG	1	0	1
1.0894	0	MSG_DEPT	0	2	1
1.1151	0	RECV_MSG	1	0	2
1.1526	1	SEND_MSG	1	0	13
1.17	1	SEND_MSG	1	0	14
1.2355	1	SEND_MSG	1	0	15
...	...	...	...	...	...
9.2051	1	SEND_MSG	1	0	103
9.2854	1	SEND_MSG	1	0	104
9.2959	0	RECV_MSG	1	0	95
9.2959	0	MSG_DEPT	0	2	83
9.2981	1	SEND_MSG	1	0	105
9.327	0	RECV_MSG	1	0	96
9.4192	0	RECV_MSG	1	0	97
9.4208	1	SEND_MSG	1	0	106
9.4393	1	SEND_MSG	1	0	107
9.4638	1	SEND_MSG	1	0	108
9.5373	1	SEND_MSG	1	0	109
9.5839	0	RECV_MSG	1	0	98
9.6362	0	RECV_MSG	1	0	99
9.6499	0	RECV_MSG	1	0	100
9.8514	1	SEND_MSG	1	0	110
9.8721	1	SEND_MSG	1	0	111
9.9032	1	SEND_MSG	1	0	112
9.9139	1	SEND_MSG	1	0	113
9.9434	1	SEND_MSG	1	0	114
9.9484	0	RECV_MSG	1	0	101
9.9484	0	MSG_DEPT	0	2	87
END	---	---	---	---	---

Duration=10, Clients count=1, Client's $\lambda=4$, Servers count=1, Q=8



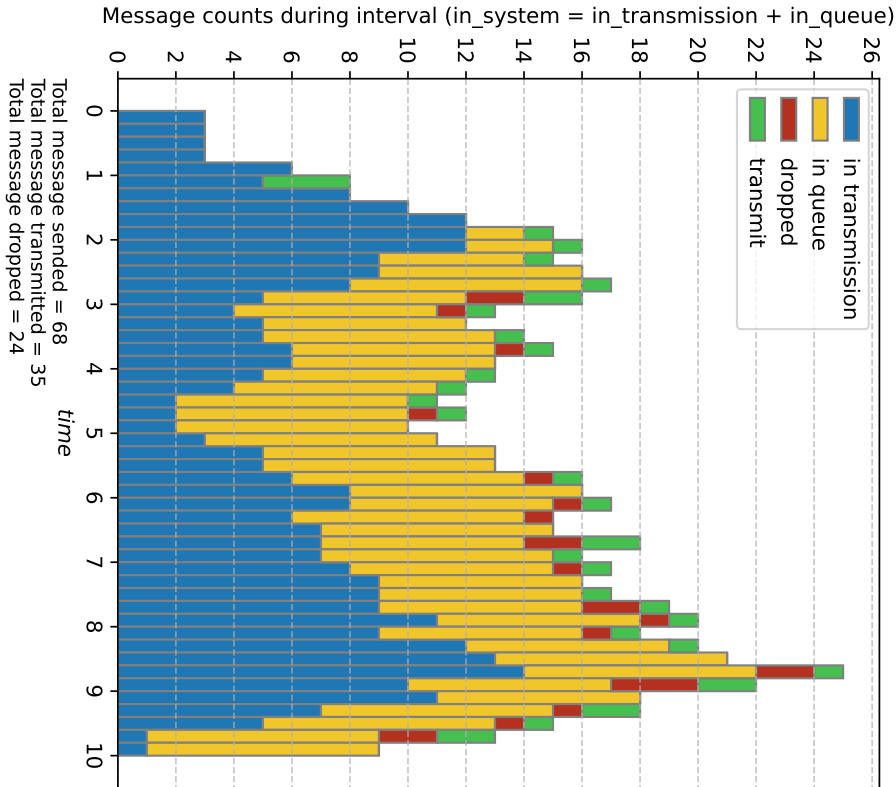
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.2683	1	SEND_MSG	1	0	1
0.3454	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.2683	0	RECV_MSG	1	0	1
1.2683	0	MSG_DEPT	0	2	1
1.3454	0	RECV_MSG	1	0	2
1.3454	0	MSG_DEPT	0	2	2
1.6892	1	SEND_MSG	1	0	3
1.7808	1	SEND_MSG	1	0	4
1.8097	1	SEND_MSG	1	0	5
2.2596	1	SEND_MSG	1	0	6
2.3843	1	SEND_MSG	1	0	7
2.5221	1	SEND_MSG	1	0	8
2.5296	1	SEND_MSG	1	0	9
2.6892	0	RECV_MSG	1	0	3
2.6892	0	MSG_DEPT	0	2	3
2.7206	1	SEND_MSG	1	0	10
2.7808	0	RECV_MSG	1	0	4
2.7808	0	MSG_DEPT	0	2	4
...	...	...	...	...	...
6.1329	1	SEND_MSG	1	0	23
6.1982	0	RECV_MSG	1	0	21
6.1982	0	MSG_DEPT	0	2	15
6.4229	0	RECV_MSG	1	0	22
6.6406	1	SEND_MSG	1	0	24
6.9321	1	SEND_MSG	1	0	25
7.0142	1	SEND_MSG	1	0	26
7.1329	0	RECV_MSG	1	0	23
7.1329	0	MSG_DEPT	0	2	16
7.3729	1	SEND_MSG	1	0	27
7.4123	1	SEND_MSG	1	0	28
7.6406	0	RECV_MSG	1	0	24
7.6406	0	MSG_DEPT	0	2	17
7.9321	0	RECV_MSG	1	0	25
8.0142	0	RECV_MSG	1	0	26
8.0142	0	MSG_DEPT	0	2	18
8.3729	0	RECV_MSG	1	0	27
8.3729	0	MSG_DEPT	0	2	19
8.4123	0	RECV_MSG	1	0	28
8.4123	0	MSG_DEPT	0	2	20
9.5181	1	SEND_MSG	1	0	29
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =6, Servers count=1, Q=8



time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1788	1	SEND_MSG	1	0	1
0.2302	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1262	1	SEND_MSG	1	0	3
1.1788	0	RECV_MSG	1	0	1
1.1788	0	MSG_DEPT	0	2	1
1.1872	1	SEND_MSG	1	0	4
1.2065	1	SEND_MSG	1	0	5
1.2302	0	RECV_MSG	1	0	2
1.2302	0	MSG_DEPT	0	2	2
1.5064	1	SEND_MSG	1	0	6
1.5895	1	SEND_MSG	1	0	7
1.6814	1	SEND_MSG	1	0	8
1.6864	1	SEND_MSG	1	0	9
1.8137	1	SEND_MSG	1	0	10
1.9873	1	SEND_MSG	1	0	11
2.0257	1	SEND_MSG	1	0	12
2.1262	0	RECV_MSG	1	0	3
2.1262	0	MSG_DEPT	0	2	3
...	...	...	...	...	...
8.7022	0	RECV_MSG	1	0	36
8.7022	0	MSG_DEPT	0	2	24
8.7215	1	SEND_MSG	1	0	40
8.855	1	SEND_MSG	1	0	41
8.9016	0	RECV_MSG	1	0	37
8.9115	0	RECV_MSG	1	0	38
8.9493	1	SEND_MSG	1	0	42
9.0503	1	SEND_MSG	1	0	43
9.0955	0	RECV_MSG	1	0	39
9.0955	0	MSG_DEPT	0	2	25
9.2447	1	SEND_MSG	1	0	44
9.3329	1	SEND_MSG	1	0	45
9.3776	1	SEND_MSG	1	0	46
9.5487	1	SEND_MSG	1	0	47
9.7215	0	RECV_MSG	1	0	40
9.7215	0	MSG_DEPT	0	2	28
9.7891	1	SEND_MSG	1	0	48
9.855	0	RECV_MSG	1	0	41
9.855	0	MSG_DEPT	0	2	30
9.9242	1	SEND_MSG	1	0	49
9.9493	0	RECV_MSG	1	0	42
END	---	---	---	---	---

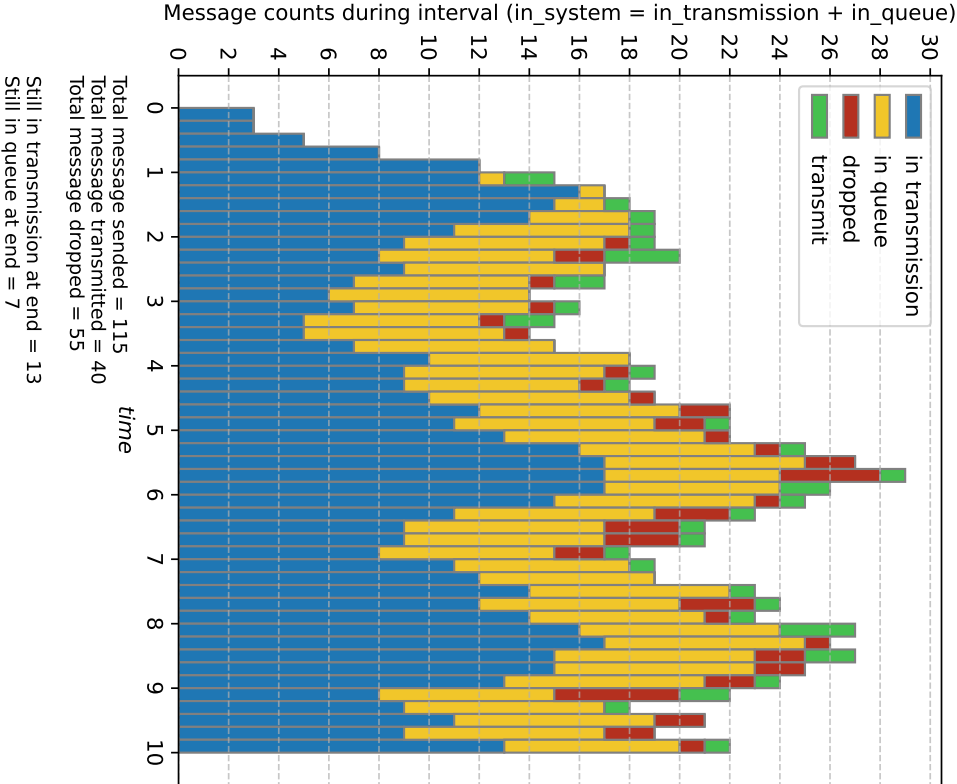
Duration=10, Clients count=1, Client's $\lambda=8$, Servers count=1, $Q=8$



Still in transmission at end = 1
Still in queue at end = 8
Average queue size = 5.8257
Average requests count in system = 12.6143
Average time in queue = 1.5106 t. u.
Maximum time in queue = 3.0816 t. u.
Average time in system (dropped included, in transmission @end ignored) = 1.7891 t. u.
Maximum time in system (dropped included, in transmission @end ignored) = 4.0816 t. u.

time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1341	1	SEND_MSG	1	0	1
0.1727	1	SEND_MSG	1	0	2
0.8446	1	SEND_MSG	1	0	3
0.8904	1	SEND_MSG	1	0	4
0.9048	1	SEND_MSG	1	0	5
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1298	1	SEND_MSG	1	0	6
1.1341	0	RECV_MSG	1	0	1
1.1341	0	MSG_DEPT	0	2	1
1.1727	0	RECV_MSG	1	0	2
1.1727	0	MSG_DEPT	0	2	2
1.1921	1	SEND_MSG	1	0	7
1.2611	1	SEND_MSG	1	0	8
1.2648	1	SEND_MSG	1	0	9
1.3603	1	SEND_MSG	1	0	10
1.4905	1	SEND_MSG	1	0	11
1.5193	1	SEND_MSG	1	0	12
1.729	1	SEND_MSG	1	0	13
1.755	1	SEND_MSG	1	0	14
...	...	...	...	...	...
8.8039	0	RECV_MSG	1	0	54
8.8811	0	RECV_MSG	1	0	55
8.8811	0	MSG_DEPT	0	2	38
8.9902	0	RECV_MSG	1	0	56
8.9902	0	MSG_DEPT	0	2	40
9.1143	1	SEND_MSG	1	0	67
9.2526	0	RECV_MSG	1	0	57
9.2526	0	MSG_DEPT	0	2	41
9.3404	0	RECV_MSG	1	0	58
9.3404	0	MSG_DEPT	0	2	44
9.3674	0	RECV_MSG	1	0	59
9.3683	0	RECV_MSG	1	0	60
9.4018	0	RECV_MSG	1	0	61
9.4018	0	MSG_DEPT	0	2	46
9.4668	0	RECV_MSG	1	0	62
9.6223	0	RECV_MSG	1	0	63
9.6223	0	MSG_DEPT	0	2	48
9.6569	0	RECV_MSG	1	0	64
9.7562	0	RECV_MSG	1	0	65
9.7562	0	MSG_DEPT	0	2	49
9.7959	0	RECV_MSG	1	0	66
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =12, Servers count=1, Q=8

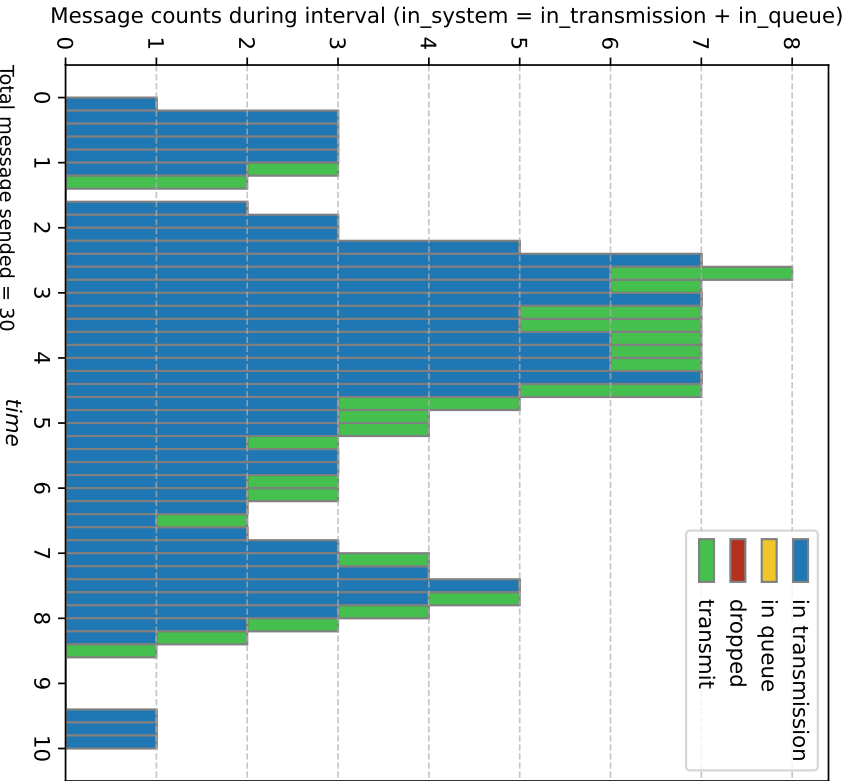


Average queue size = 6.3118
Average requests count in system = 17.0833

Average time in queue = 1.3741 t.u.
Maximum time in queue = 2.9667 t.u.
Average time in system (dropped included, in transmission @end ignored) = 1.5389 t.u.
Maximum time in system (dropped included, in transmission @end ignored) = 3.9667 t.u.

time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.0894	1	SEND_MSG	1	0	1
0.1151	1	SEND_MSG	1	0	2
0.5631	1	SEND_MSG	1	0	3
0.5936	1	SEND_MSG	1	0	4
0.6032	1	SEND_MSG	1	0	5
0.7532	1	SEND_MSG	1	0	6
0.7948	1	SEND_MSG	1	0	7
0.8407	1	SEND_MSG	1	0	8
0.8432	1	SEND_MSG	1	0	9
0.9069	1	SEND_MSG	1	0	10
0.9937	1	SEND_MSG	1	0	11
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.0129	1	SEND_MSG	1	0	12
1.0894	0	RECV_MSG	1	0	1
1.0894	0	MSG_DEPT	0	2	1
1.1151	0	RECV_MSG	1	0	2
1.1526	1	SEND_MSG	1	0	13
1.17	1	SEND_MSG	1	0	14
1.2355	1	SEND_MSG	1	0	15
...	...	...	...	...	...
9.2051	1	SEND_MSG	1	0	103
9.2854	1	SEND_MSG	1	0	104
9.2959	0	RECV_MSG	1	0	95
9.2959	0	MSG_DEPT	0	2	76
9.2981	1	SEND_MSG	1	0	105
9.327	0	RECV_MSG	1	0	96
9.4192	0	RECV_MSG	1	0	97
9.4208	1	SEND_MSG	1	0	106
9.4393	1	SEND_MSG	1	0	107
9.4638	1	SEND_MSG	1	0	108
9.5373	1	SEND_MSG	1	0	109
9.5839	0	RECV_MSG	1	0	98
9.6362	0	RECV_MSG	1	0	99
9.6499	0	RECV_MSG	1	0	100
9.8514	1	SEND_MSG	1	0	110
9.8721	1	SEND_MSG	1	0	111
9.9032	1	SEND_MSG	1	0	112
9.9139	1	SEND_MSG	1	0	113
9.9434	1	SEND_MSG	1	0	114
9.9484	0	RECV_MSG	1	0	101
9.9484	0	MSG_DEPT	0	2	77
END	---	---	---	---	---

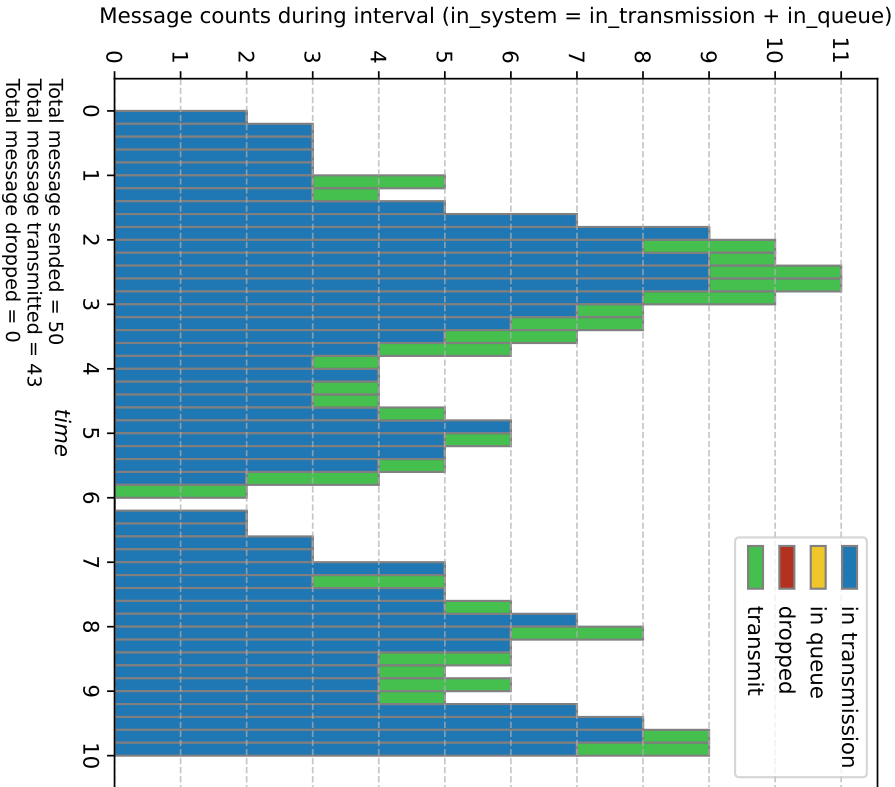
Duration=10, Clients count=1, Client's $\lambda=4$, Servers count=3, Q=8



Total message sendend = 30
Total message transmitted = 29
Total message dropped = 0
Still in transmission at end = 1
Still in queue at end = 0
Average queue size = 0.0000
Average requests count in system = 2.9482
Average time in queue = 0.0000 t.u.
Maximum time in queue = 0.0000 t.u.
Average time in system (dropped included, in transmission @end ignored) = 1.0000 t.u.
Maximum time in system (dropped included, in transmission @end ignored) = 1.0000 t.u.

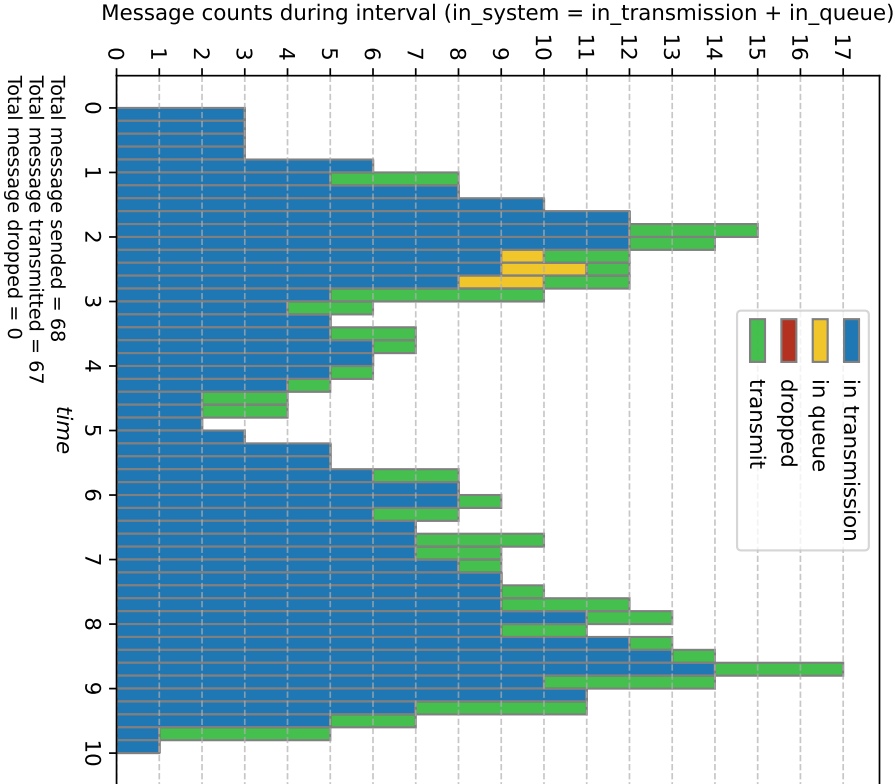
time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.2683	1	SEND_MSG	1	0	1
0.3454	1	SEND_MSG	1	0	2
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.2683	0	RECV_MSG	1	0	1
1.2683	0	MSG_DEPT	0	2	1
1.3454	0	RECV_MSG	1	0	2
1.3454	0	MSG_DEPT	0	2	2
1.6892	1	SEND_MSG	1	0	3
1.7808	1	SEND_MSG	1	0	4
1.8097	1	SEND_MSG	1	0	5
2.2596	1	SEND_MSG	1	0	6
2.3843	1	SEND_MSG	1	0	7
2.5221	1	SEND_MSG	1	0	8
2.5296	1	SEND_MSG	1	0	9
2.6892	0	RECV_MSG	1	0	3
2.6892	0	MSG_DEPT	0	2	3
2.7206	1	SEND_MSG	1	0	10
2.7808	0	RECV_MSG	1	0	4
2.7808	0	MSG_DEPT	0	2	4
...	...	...	...	...	...
6.1982	0	MSG_DEPT	0	2	21
6.4229	0	RECV_MSG	1	0	22
6.4229	0	MSG_DEPT	0	3	22
6.6406	1	SEND_MSG	1	0	24
6.9321	1	SEND_MSG	1	0	25
7.0142	1	SEND_MSG	1	0	26
7.1329	0	RECV_MSG	1	0	23
7.1329	0	MSG_DEPT	0	2	23
7.3729	1	SEND_MSG	1	0	27
7.4123	1	SEND_MSG	1	0	28
7.6406	0	RECV_MSG	1	0	24
7.6406	0	MSG_DEPT	0	2	24
7.9321	0	RECV_MSG	1	0	25
7.9321	0	MSG_DEPT	0	3	25
8.0142	0	RECV_MSG	1	0	26
8.0142	0	MSG_DEPT	0	2	26
8.3729	0	RECV_MSG	1	0	27
8.3729	0	MSG_DEPT	0	2	27
8.4123	0	RECV_MSG	1	0	28
8.4123	0	MSG_DEPT	0	2	28
9.5181	1	SEND_MSG	1	0	29
END	...	...	...	...	...

Duration=10, Clients count=1, Client's λ =6, Servers count=3, Q=8



time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1788	1	SEND_MSG	1	0	1
0.2302	1	SEND_MSG	1	0	2
1.0	0	RCV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1262	1	SEND_MSG	1	0	3
1.1788	0	RCV_MSG	1	0	1
1.1788	0	MSG_DEPT	0	2	1
1.1872	1	SEND_MSG	1	0	4
1.2065	1	SEND_MSG	1	0	5
1.2302	0	RCV_MSG	1	0	2
1.2302	0	MSG_DEPT	0	2	2
1.5064	1	SEND_MSG	1	0	6
1.5895	1	SEND_MSG	1	0	7
1.6814	1	SEND_MSG	1	0	8
1.6864	1	SEND_MSG	1	0	9
1.8137	1	SEND_MSG	1	0	10
1.9873	1	SEND_MSG	1	0	11
2.0257	1	SEND_MSG	1	0	12
2.1262	0	RCV_MSG	1	0	3
2.1262	0	MSG_DEPT	0	2	3
...	...	...	...	...	...
8.855	1	SEND_MSG	1	0	41
8.9016	0	RCV_MSG	1	0	37
8.9016	0	MSG_DEPT	0	3	37
8.9115	0	RCV_MSG	1	0	38
8.9115	0	MSG_DEPT	0	4	38
8.9493	1	SEND_MSG	1	0	42
9.0503	1	SEND_MSG	1	0	43
9.0955	0	RCV_MSG	1	0	39
9.0955	0	MSG_DEPT	0	2	39
9.2447	1	SEND_MSG	1	0	44
9.3329	1	SEND_MSG	1	0	45
9.3776	1	SEND_MSG	1	0	46
9.5487	1	SEND_MSG	1	0	47
9.7215	0	RCV_MSG	1	0	40
9.7215	0	MSG_DEPT	0	2	40
9.7891	1	SEND_MSG	1	0	48
9.855	0	RCV_MSG	1	0	41
9.855	0	MSG_DEPT	0	2	41
9.9242	1	SEND_MSG	1	0	49
9.9493	0	RCV_MSG	1	0	42
9.9493	0	MSG_DEPT	0	3	42
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =8, Servers count=3, Q=8

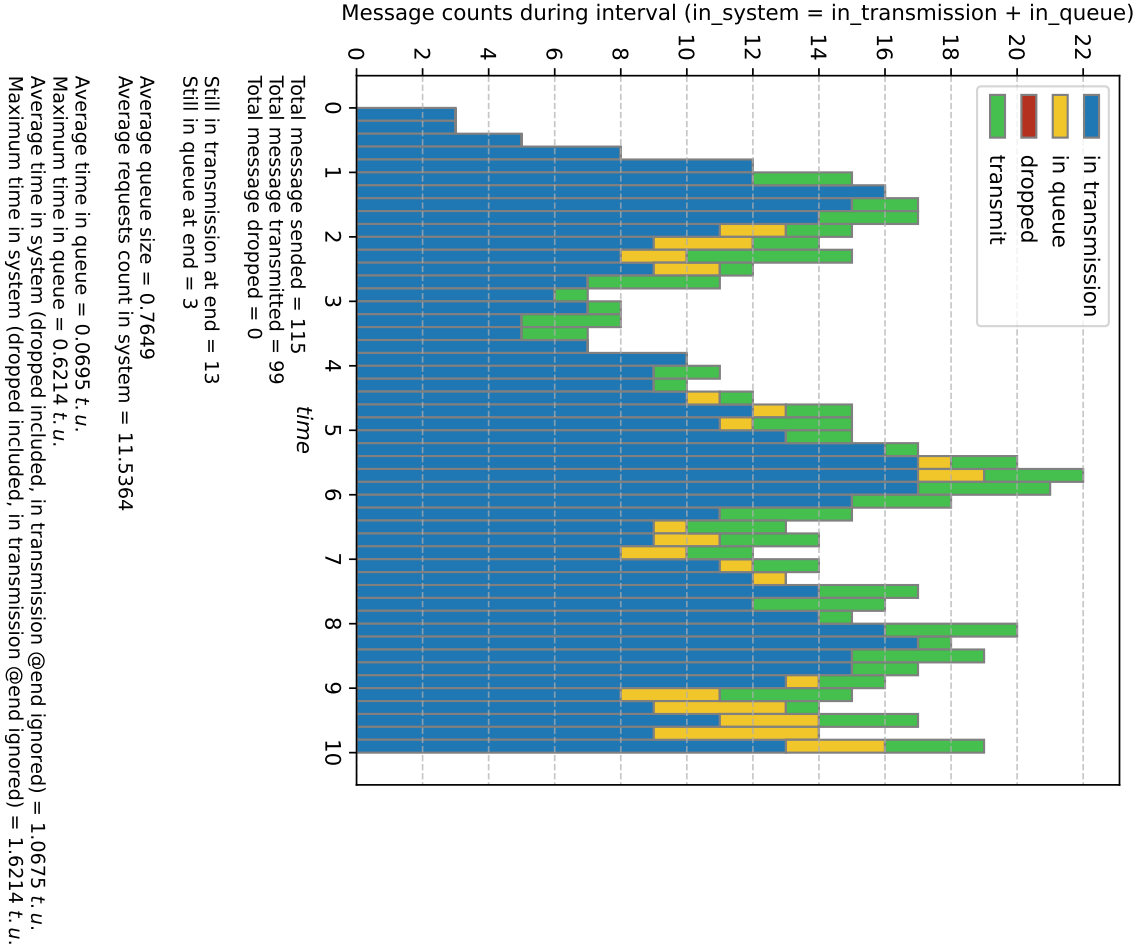


Average queue size = 0.1305
Average requests count in system = 6.9190

Average time in queue = 0.0195 t. u.
Maximum time in queue = 0.2385 t. u.
Average time in system (dropped included, in transmission @end ignored) = 1.0195 t. u.
Maximum time in system (dropped included, in transmission @end ignored) = 1.2385 t. u.

time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.1341	1	SEND_MSG	1	0	1
0.1727	1	SEND_MSG	1	0	2
0.8446	1	SEND_MSG	1	0	3
0.8904	1	SEND_MSG	1	0	4
0.9048	1	SEND_MSG	1	0	5
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.1298	1	SEND_MSG	1	0	6
1.1341	0	RECV_MSG	1	0	1
1.1341	0	MSG_DEPT	0	2	1
1.1727	0	RECV_MSG	1	0	2
1.1727	0	MSG_DEPT	0	2	2
1.1921	1	SEND_MSG	1	0	7
1.2611	1	SEND_MSG	1	0	8
1.2648	1	SEND_MSG	1	0	9
1.3603	1	SEND_MSG	1	0	10
1.4905	1	SEND_MSG	1	0	11
1.5193	1	SEND_MSG	1	0	12
1.729	1	SEND_MSG	1	0	13
1.755	1	SEND_MSG	1	0	14
...	...	...	...	...	...
9.1143	1	SEND_MSG	1	0	67
9.2526	0	RECV_MSG	1	0	57
9.2526	0	MSG_DEPT	0	2	57
9.3404	0	RECV_MSG	1	0	58
9.3404	0	MSG_DEPT	0	2	58
9.3674	0	RECV_MSG	1	0	59
9.3674	0	MSG_DEPT	0	3	59
9.3683	0	RECV_MSG	1	0	60
9.3683	0	MSG_DEPT	0	4	60
9.4018	0	RECV_MSG	1	0	61
9.4018	0	MSG_DEPT	0	2	61
9.4668	0	RECV_MSG	1	0	62
9.4668	0	MSG_DEPT	0	3	62
9.6223	0	RECV_MSG	1	0	63
9.6223	0	MSG_DEPT	0	2	63
9.6569	0	RECV_MSG	1	0	64
9.6569	0	MSG_DEPT	0	3	64
9.7562	0	RECV_MSG	1	0	65
9.7562	0	MSG_DEPT	0	2	65
9.7959	0	RECV_MSG	1	0	66
9.7959	0	MSG_DEPT	0	4	66
END	---	---	---	---	---

Duration=10, Clients count=1, Client's λ =12, Servers count=3, Q=8



time	node	event	src	dst	msgID
0.0	1	SEND_MSG	1	0	0
0.0894	1	SEND_MSG	1	0	1
0.1151	1	SEND_MSG	1	0	2
0.5631	1	SEND_MSG	1	0	3
0.5936	1	SEND_MSG	1	0	4
0.6032	1	SEND_MSG	1	0	5
0.7532	1	SEND_MSG	1	0	6
0.7948	1	SEND_MSG	1	0	7
0.8407	1	SEND_MSG	1	0	8
0.8432	1	SEND_MSG	1	0	9
0.9069	1	SEND_MSG	1	0	10
0.9937	1	SEND_MSG	1	0	11
1.0	0	RECV_MSG	1	0	0
1.0	0	MSG_DEPT	0	2	0
1.0129	1	SEND_MSG	1	0	12
1.0894	0	RECV_MSG	1	0	1
1.0894	0	MSG_DEPT	0	2	1
1.1151	0	RECV_MSG	1	0	2
1.1151	0	MSG_DEPT	0	3	2
1.1526	1	SEND_MSG	1	0	13
1.17	1	SEND_MSG	1	0	14
...	...	...	...	...	...
9.327	0	RECV_MSG	1	0	96
9.4192	0	RECV_MSG	1	0	97
9.4192	0	MSG_DEPT	0	3	93
9.4192	0	MSG_DEPT	0	4	94
9.4208	1	SEND_MSG	1	0	106
9.4393	1	SEND_MSG	1	0	107
9.4638	1	SEND_MSG	1	0	108
9.5373	1	SEND_MSG	1	0	109
9.5839	0	RECV_MSG	1	0	98
9.5839	0	MSG_DEPT	0	3	95
9.6362	0	RECV_MSG	1	0	99
9.6499	0	RECV_MSG	1	0	100
9.8514	1	SEND_MSG	1	0	110
9.8721	1	SEND_MSG	1	0	111
9.9032	1	SEND_MSG	1	0	112
9.9139	1	SEND_MSG	1	0	113
9.9434	1	SEND_MSG	1	0	114
9.9484	0	RECV_MSG	1	0	101
9.9484	0	MSG_DEPT	0	2	96
9.9484	0	MSG_DEPT	0	3	97
9.9484	0	MSG_DEPT	0	4	98
END	---	---	---	---	---